MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 7 (4-8 Oct 2026) - Period 1	Ch 8 - 8.1 Programming concepts

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 7 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Ch 7 review test: trace tables strong; ~35% weak at stating an algorithm's purpose. Today: start from a 3-line trace to link algorithms to real code.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students declare and use variables and constants, choose meaningful identifiers, and run a first Python program from Cambridge pseudocode. Syllabus 0478 8.1.1; AO1, AO2.

Success criteria — *what will successful learning look / sound like?*

I can explain variable vs constant; I can write DECLARE and CONSTANT statements in pseudocode; I can translate them into Python and run the program without errors.

Key knowledge / skill / understanding:

Variable = named memory location that can change; a constant stays fixed while the program runs.

Likely misconception / access or challenge need:

'=' in Python is assignment, not equality; a constant is just a variable written in capitals. EAL: 'declare', 'assign', 'identifier'.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Think-pair-share **Time:** 8 min

What students will do:

Trace a 3-line pseudocode snippet (Score <- 5; Score <- Score + 2; OUTPUT Score). Predict output alone, compare with a partner, then share why Score changed but HourlyRate cannot.

Evidence tool: Mini-whiteboards

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Slide with snippet; mini-whiteboards; Ch 8 key-term card.

Support (low performers / SEN):

Key-term card with picture of 'labelled boxes' for variables.

Stretch (fast learners / gifted):

Explain why constants make code easier to update.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 22 min

What students will do:

Follow a worked example: write DECLARE/CONSTANT pseudocode for a 'school canteen order' (Price, Quantity, VAT_RATE), then code it in Python (IDLE/Thonny), run and fix errors.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.1; Python 3 + Thonny; starter file canteen.py.

Support (low performers / SEN):

Starter file with comments showing where each line goes.

Stretch (fast learners / gifted):

Add a second constant and output a formatted total with 2 decimal places.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Gamification **Time:** 7 min

What students will do:

Quizizz 'Variable or constant?' and 'Good or bad identifier?' (e.g. x, TotalCost, 2ndName, VAT_RATE).

Evidence tool: Digital quiz (Forms / Kahoot / Quizizz)

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Quizizz with 10 items.

Support (low performers / SEN):

Read-aloud mode; picture prompts.

Stretch (fast learners / gifted):

Bonus: rewrite 3 bad identifiers correctly.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: a phone contact list - name stays, number may change. Model on the board: DECLARE Price : REAL, CONSTANT VAT_RATE <- 0.15, then the Python equivalent. Think aloud on choosing identifiers. Show Thonny: run, read the error message, fix. Key words: variable, constant, identifier, assignment.

What is new or innovative in this lesson? *(ETEC looks for this)*

Every program starts as Cambridge pseudocode, then becomes Python, so students practise the exam notation and real coding side by side.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Spaces in identifiers; using = in pseudocode instead of <-; forgetting quotes for strings; NameError from typos; students copying without running. Likely: 'a constant is a variable in capitals' - ask what stops it changing.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket (Forms, 3 Qs): define constant; correct a faulty DECLARE line; name one benefit of meaningful identifiers. Time for learners' questions.

Homework / platform *(e.g. Madrasati)*

Teams/Madrasati: coursebook 8.1 questions 1-3; write pseudocode + Python for a 'bus fare' program with one constant.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Canteen context made sense to all; EAL learners used the key-term card well. 2 of 3 classes ran the program by minute 30. Exit tickets: 80% defined constant correctly.

What will I change next time?

Install-check Thonny before class (3 PCs in 10BR2 missing it). Show the <- vs = difference twice; it caused most errors.

Students needing follow-up:

10BR3: 4 students still mixing up assignment and equality; recent arrivals need the bilingual key-term card again.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☑2 ☐3 ☐C ☐H

Create	Discussion	Pair / group	Learner choice	Self-assessment	Peer assessment	SEN / low	Gifted
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 7 (4-8 Oct 2026) - Period 2	Ch 8 - 8.2 Data types

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 7 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Exit ticket: 80% defined constant; <- vs = confusion remains. Today: 3-min warm-up on assignment, then data types.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students identify and use INTEGER, REAL, CHAR, STRING and BOOLEAN, choose a suitable type for given data, and convert types in Python (int(), float(), str()). Syllabus 8.1.2; AO1, AO2.

Success criteria — *what will successful learning look / sound like?*

I can choose the correct data type for 8 data items and justify it; I can explain why input() needs converting before arithmetic; my program converts and calculates correctly.

Key knowledge / skill / understanding:

Data type decides what values are stored and which operations work; Python input() always returns a string.

Likely misconception / access or challenge need:

Phone numbers are integers; '5' + '5' = 10; REAL and INTEGER are interchangeable.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Cooperative learning **Time:** 10 min

What students will do:

Card sort in groups of 3: 16 data cards (age, price, grade letter, passport number, isRegistered...) placed under INTEGER, REAL, CHAR, STRING, BOOLEAN with a reason.

Evidence tool: Observation checklist

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Card-sort sets; 5 type headers.

Support (low performers / SEN):

Mixed-language groups; examples on each header.

Stretch (fast learners / gifted):

Find 2 items that could fit 2 types and argue the best choice.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Inquiry / investigation **Time:** 20 min

What students will do:

Investigate in Python: what do type('5'), '5'+'5', int('5')+5, float ('2.5'), int('2.5') do? Record results in a table, then fix a broken 'average mark' program.

Evidence tool: Portfolio / notebook work

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Python/Thonny; investigation sheet; broken average.py.

Support (low performers / SEN):

Investigation sheet with first 2 rows done.

Stretch (fast learners / gifted):

Explain the error from int('2.5') and write code that avoids it.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Debate **Time:** 7 min

What students will do:

'Should a phone number be stored as an INTEGER?' Two teams argue, using leading zeros and +966 as evidence; class votes.

Evidence tool: Oral questioning

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Debate prompt; example numbers.

Support (low performers / SEN):

Sentence starters.

Stretch (fast learners / gifted):

Link the answer to validation from Chapter 7.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Warm-up: 3 lines with <- vs =. Hook: a form with name, age, height, grade letter and 'Paid?' - what kind of data is each? Model DECLARE with each type and Python type() in the shell. Show input() returning a string and int(input()).

What is new or innovative in this lesson? *(ETEC looks for this)*

Students investigate the computer's behaviour themselves and record evidence before the rule is given; a short debate turns a typical exam trap into reasoning.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Treating numbers with leading zeros as INTEGER; CHAR vs STRING confusion; TypeError from adding str and int; float precision surprise (0.1+0.2). Likely: 'Python has no CHAR' - explain how exams still use it.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit check: choose the data type for 5 new items and explain one; predict output of '3' * 2. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.2 activity questions; complete a data dictionary (identifier, type, example) for a school library system.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Card sort sparked useful talk across language groups; Saudi, Egyptian and Indian students compared phone formats, which made the debate real. Investigation table done by 85%.

What will I change next time?

Give the investigation table on paper; switching windows slowed weaker students. Leave float precision as a stretch only.

Students needing follow-up:

10BR1: 3 students unsure about CHAR vs STRING; one SEN student needs the type-colour poster at the desk.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H

Create	Discussion	Pair / group	Learner choice	Self-assessment	Peer assessment	SEN / low	Gifted
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 8 (11-15 Oct 2026) - Period 1	Ch 8 - Programming task 8.1 (input and output)

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 8 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Data dictionary homework: 75% correct types; casting still shaky. Today: apply types in a full input-process-output program (Programming task 8.1).

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use INPUT and OUTPUT with prompts and clear messages to complete Programming task 8.1, applying variables, constants and data types. Syllabus 8.1.1-8.1.3; AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

My program asks clear prompts, converts inputs to the right type, processes them and outputs a readable message; my pseudocode matches my Python code.

Key knowledge / skill / understanding:

Input-process-output model; OUTPUT can join text and variables; prompts guide the user.

Likely misconception / access or challenge need:

OUTPUT and INPUT must be on one line; print adds spaces automatically everywhere.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Discussion / dialogu **Time:** 6 min

What students will do:

Look at two versions of the same program (one with no prompts/messages, one user-friendly). Discuss what makes output clear for any user, including EAL users.

Evidence tool: Oral questioning

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Two program demos.

Support (low performers / SEN):

Picture of input-process-output diagram.

Stretch (fast learners / gifted):

Suggest an accessibility improvement.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Project-based task **Time:** 24 min

What students will do:

Programming task 8.1: plan with an IPO table, write pseudocode, then code in Python a 'travel cost calculator' (distance, fuel price, passengers -> cost per person). Test with 3 sets of data.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) Programming task 8.1; IPO table template; test table.

Support (low performers / SEN):

IPO table pre-filled for inputs; code skeleton.

Stretch (fast learners / gifted):

Add currency choice (SAR/USD) using a constant exchange rate.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Cooperative learning **Time:** 7 min

What students will do:

Pair test swap: run partner's program with normal, boundary and silly data; record results in a test table and give one improvement.

Evidence tool: Peer-assessment checklist

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Test table template.

Support (low performers / SEN):

Test data given.

Stretch (fast learners / gifted):

Suggest validation to stop silly data.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: a ticket machine that says nothing - how do you use it? Model IPO table, then pseudocode: INPUT Distance; OUTPUT "Cost per person: ", Cost. Translate to Python with float(input('...')) and print(). Share success criteria.

What is new or innovative in this lesson? *(ETEC looks for this)*

Students plan with an IPO table and test with three kinds of data, following the full design-code-test cycle used in Paper 2 scenario questions.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

No type conversion; prompts missing; OUTPUT joined with + causing TypeError; division by zero passengers. Likely: 'it works with my numbers' - push for test data.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: write pseudocode lines for input of Name and output of a greeting; name one test you ran. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Finish task 8.1 and upload code + test table. Read coursebook 8.4 arithmetic operators.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Travel context worked well - students used routes to their home cities. Peer testing found real bugs (zero passengers). 80% completed with test table.

What will I change next time?

Model the test table with one example row first; several students tested only 'normal' data.

Students needing follow-up:

10BR2: 4 students did not finish coding; extra lab time offered. Gifted students: currency extension done by 5.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 8 (11-15 Oct 2026) - Period 2	Ch 8 - 8.4 Arithmetic operators and 8.5 Sequence

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 8 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Task 8.1 uploads: 80%; test tables weak on boundary data. Today: arithmetic operators (incl. MOD, DIV, ^) and why order of statements matters.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use arithmetic operators + - * / ^ MOD DIV in pseudocode and Python (**, //, %), apply operator precedence, and explain sequence. Syllabus 8.1.4(a), 8.1.4(f); AO1, AO2.

Success criteria — *what will successful learning look / sound like?*

I can evaluate expressions with MOD and DIV; I can convert pseudocode operators into Python; I can reorder jumbled statements into a correct sequence and explain why.

Key knowledge / skill / understanding:

DIV = whole-number quotient; MOD = remainder; precedence: brackets, ^, * / , + -; sequence = statements run in order.

Likely misconception / access or challenge need:

MOD is a percentage; / and DIV give the same result; order of lines doesn't matter.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Problem solving **Time:** 10 min

What students will do:

Calculate by hand (no calculators, as in Paper 2): 17 DIV 5, 17 MOD 5, 2^3, 10 - 4 / 2. Then convert 'seconds to minutes and seconds' using DIV and MOD.

Evidence tool: Mini-whiteboards

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Expression slide; whiteboards.

Support (low performers / SEN):

Number line and counters for DIV/MOD.

Stretch (fast learners / gifted):

Write an expression to check if a number is even.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Jigsaw **Time:** 20 min

What students will do:

Expert groups master one program (time converter, change-giver, even/odd checker), write its pseudocode and Python, then teach it to a mixed group.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Coursebook (CUP 9781108915144) 8.4-8.5; expert cards; Python.

Support (low performers / SEN):

Expert card with partially written pseudocode.

Stretch (fast learners / gifted):

Change-giver for SAR notes 500/100/50/10/5/1 using DIV and MOD.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Gamification **Time:** 7 min

What students will do:

Parsons puzzle race: reorder 8 jumbled lines of a 'shopping total' program into the correct sequence; justify one move.

Evidence tool: Exit ticket

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Parsons puzzle (paper strips or online).

Support (low performers / SEN):

Fewer lines (5).

Stretch (fast learners / gifted):

Find the line that can move without changing output.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: 125 seconds on a stopwatch - how many minutes and seconds? Model DIV and MOD, then Python // and %. Show precedence with brackets. Explain jigsaw roles and timing.

What is new or innovative in this lesson? *(ETEC looks for this)*

Change-giver uses real Saudi riyal notes; jigsaw makes each student an expert who teaches a program.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Using / instead of DIV; ^ typed in Python (bitwise XOR, not power); precedence errors; statements in wrong order using a variable before input. Likely: '17 MOD 5 = 3.4'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: evaluate 29 DIV 4 and 29 MOD 4; reorder 4 lines; one question for the teacher.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.4 questions; write a program that converts minutes to hours and minutes using DIV and MOD.

5 REFLECT — After the lesson *Complete within a day. This is reflective-practice evidence and feeds your next Starting point.*

What worked — and how do I know?

Riyal change-giver was a hit and linked to real shopping. Hand calculation exposed MOD misconceptions early. 85% correct on exit ticket.

What will I change next time?

Warn about ^ in Python (bitwise XOR) before coding; it caused most wrong outputs. Give the Parsons strips on paper for faster sorting.

Students needing follow-up:

10BR3: 3 students still treat MOD as division; give extra counters practice. Inform parents of 2 missing uploads.

COVERAGE CHECK *Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.*

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H

Create	Discussion	Pair / group	Learner choice	Self-assessment	Peer assessment	SEN / low	Gifted
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 9 (18-22 Oct 2026) - Period 1	Ch 8 - 8.6 Selection: IF statements

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 9 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Exit ticket: 85% DIV/MOD correct; ^ vs ** errors in code. Today: relational operators and IF...THEN...ELSE...ENDIF.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use relational operators (= < <= > >= <>) and write IF...THEN...ELSE...ENDIF in pseudocode and Python if/elif/else to make decisions. Syllabus 8.1.4(b), 8.1.4(f); AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I can write a correct IF statement with ELSE in Cambridge pseudocode; I can code it in Python with correct indentation; I test both branches.

Key knowledge / skill / understanding:

Selection chooses a path based on a Boolean condition; <> means not equal (!= in Python); indentation defines blocks in Python.

Likely misconception / access or challenge need:

= compares in Python (it assigns); ELSE needs a condition; only one IF can be used.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Storytelling **Time:** 6 min

What students will do:

A short story: a student arrives at the school gate - IF before 7:15 THEN 'on time' ELSE 'late slip'. Students turn the story into a flowchart decision.

Evidence tool: Mini-whiteboards

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Story slide; flowchart symbols.

Support (low performers / SEN):

Flowchart with decision box drawn.

Stretch (fast learners / gifted):

Add a third outcome (very late).

ACTIVITY 2 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 22 min

What students will do:

Worked example then practice: grade calculator (mark -> A*, A, B, C, U) in pseudocode with nested IF/ELSE and Python if/elif/else. Test each boundary mark.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.6; Python; boundary test table.

Support (low performers / SEN):

Template with first two branches written.

Stretch (fast learners / gifted):

Validate input is 0-100 before grading.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Think-pair-share **Time:** 8 min

What students will do:

Spot the error: 4 short IF snippets with faults (missing ENDIF, = vs ==, wrong boundary, bad indentation). Pairs fix and explain.

Evidence tool: Peer-assessment checklist

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Error-spotting sheet.

Support (low performers / SEN):

Errors highlighted, students explain fix.

Stretch (fast learners / gifted):

Write one buggy snippet for another pair.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 7 min

Hook: traffic light decision. Model relational operators, then IF...THEN...ELSE...ENDIF with 2-space indentation as in the syllabus, and Python if/elif/else. Think aloud on testing boundaries (e.g. 69, 70, 71).

What is new or innovative in this lesson? *(ETEC looks for this)*

Boundary testing taught explicitly with test tables, mirroring how Paper 2 marks careful condition writing.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Wrong boundary (> vs >=); missing colon or indentation; using = in conditions; order of elif tests wrong so A* never reached. Likely: 'my program always says U'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: write pseudocode IF for 'age >= 18 OUTPUT Adult ELSE Minor'; give 3 boundary test values. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.6 IF questions; program: input temperature, output 'Hot', 'Warm' or 'Cold'.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Gate story was instantly relatable; grade calculator engaged all. Error-spotting pairs mixing language groups worked well. 80% tested all boundaries.

What will I change next time?

Show the order-of-elif problem with a live failing example earlier. Provide boundary tables on paper.

Students needing follow-up:

10BR1, 10BR3: about 3 students per class still mix = and ==; quick retrieval starter next lesson.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H
Create	**Discussion**	**Pair / group**	**Learner choice**	**Self-assessment**	**Peer assessment**	**SEN / low**	**Gifted**
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 9 (18-22 Oct 2026) - Period 2	Ch 8 - 8.6 Selection: CASE statements

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 9 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Exit ticket: 80% wrote correct IF; boundary choices weak in 10BR3. Today: CASE...OF...OTHERWISE...ENDCASE and when to choose CASE over IF.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students write CASE OF ... OTHERWISE ... ENDCASE in pseudocode, implement it in Python (match-case or if/elif), and justify CASE vs IF. Syllabus 8.1.4(b); AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I can write a CASE statement with an OTHERWISE clause; I can code it in Python; I can explain when CASE is clearer than nested IF.

Key knowledge / skill / understanding:

CASE selects one branch from many values of one variable; OTHERWISE must be last; Python 3.10+ match-case or if/elif.

Likely misconception / access or challenge need:

CASE can test any condition like IF; OTHERWISE is optional so never needed.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Role play **Time:** 7 min

What students will do:

'Customer service menu' role play: one student is the menu (1=Balance, 2=Top-up, 3=Help, other='Invalid'), others call in; class records the branches.

Evidence tool: Observation checklist

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Menu card; role cards.

Support (low performers / SEN):

Role card with script.

Stretch (fast learners / gifted):

Add a hidden option and discuss OTHERWISE.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 22 min

What students will do:

Rewrite a nested-IF 'day of week' program as CASE in pseudocode, then code a canteen menu (CASE on item code) in Python with match-case or if/elif.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.6 CASE; syllabus pseudocode guide p.44; Python 3.10+.

Support (low performers / SEN):

Side-by-side IF vs CASE template.

Stretch (fast learners / gifted):

Combine CASE with a loop later: menu repeats until 'Q'.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Discussion / dialogu **Time:** 7 min

What students will do:

Pairs decide: IF or CASE? for 5 scenarios (grade boundaries, menu choice, password check, month number, shipping zones) and justify.

Evidence tool: Exit ticket

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Scenario cards.

Support (low performers / SEN):

Scenarios with picture hints.

Stretch (fast learners / gifted):

Explain why ranges suit IF better.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 7 min

Retrieval: = vs == quick check. Hook: a phone menu 'press 1, press 2'. Model CASE OF Choice ... OTHERWISE OUTPUT "Invalid" ENDCASE (syllabus layout), then Python match-case and the if/elif equivalent.

What is new or innovative in this lesson? *(ETEC looks for this)*

Role-play models program flow physically before coding; students judge the best control structure, building AO3 evaluation.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

OTHERWISE not last; using ranges in CASE; Python version without match-case (fallback to if/elif); missing default. Likely: 'CASE is just shorter IF'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit check: write a 4-branch CASE in pseudocode; one sentence on CASE vs IF. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.6 CASE questions; program: input month number 1-12, output the season (handle invalid).

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Role play made OTHERWISE obvious. Students debated Hijri vs Gregorian months for the homework - good cultural link.

What will I change next time?

Check Python version in the lab (2 PCs below 3.10, no match-case); teach if/elif fallback first.

Students needing follow-up:

Students who missed the IF lesson (approx. 2 per class) need the worked example sheet.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☑C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☑3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 10 (25-29 Oct 2026) - Period 1	Ch 8 - 8.6 Boolean operators + Task 8.2

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 10 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

CASE exit check: 85% correct; choosing CASE vs IF improving. Today: AND, OR, NOT and Programming task 8.2.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use logical operators AND, OR, NOT in conditions, evaluate them with truth tables, and complete Programming task 8.2. Syllabus 8.1.4(f); links to Topic 10 logic; AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I can evaluate compound conditions; I can write conditions with AND/OR/NOT in pseudocode and Python; my task 8.2 program handles all cases.

Key knowledge / skill / understanding:

AND needs both true; OR needs at least one; NOT reverses; brackets control order.

Likely misconception / access or challenge need:

'x > 5 AND < 10' is valid; OR means exactly one is true.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Gamification **Time:** 8 min

What students will do:

'Stand up if TRUE': teacher reads compound conditions about the class (wears glasses AND born in October...); students stand or sit, then write the truth table.

Evidence tool: Oral questioning

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Condition list; truth table slide.

Support (low performers / SEN):

Truth table with 2 rows filled.

Stretch (fast learners / gifted):

Write a condition that is TRUE for exactly 3 students.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Project-based task **Time:** 22 min

What students will do:

Programming task 8.2 (coursebook): plan, write pseudocode and Python using compound conditions, e.g. ticket pricing with age AND student status; test with a test table.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) Programming task 8.2; test table.

Support (low performers / SEN):

Pseudocode skeleton with conditions to complete.

Stretch (fast learners / gifted):

Add NOT and explain an equivalent condition without it.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Cooperative learning **Time:** 7 min

What students will do:

Pairs swap test tables and check each other's expected vs actual results; flag one missed test case.

Evidence tool: Peer-assessment checklist

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Peer checklist.

Support (low performers / SEN):

Checklist with example.

Stretch (fast learners / gifted):

Write the missing test case.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: a theme-park ride - you can ride IF height >= 140 AND age >= 10. Model AND/OR/NOT truth tables, then pseudocode IF Age >= 10 AND Height >= 140 THEN ... and Python and/or/not. Link to logic gates (Topic 10).

What is new or innovative in this lesson? *(ETEC looks for this)*

Whole-body truth-table game makes logic visible; explicit link to logic gates topic builds cross-topic understanding.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Writing incomplete conditions (Age > 5 AND < 10); OR used instead of AND; missing brackets with NOT. Likely: 'OR means one or the other but not both'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: evaluate 3 compound conditions; fix 'Mark > 50 AND < 70'. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Finish task 8.2 and upload code + test table. Read coursebook 8.7 iteration (count-controlled).

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Standing game got everyone involved, including quieter students; 80% completed task 8.2. Compound condition errors dropped after the exit ticket.

What will I change next time?

Keep game to 6 minutes; extend testing time. Pre-teach the 'incomplete condition' error.

Students needing follow-up:

10BR2: 3 students need help with planning before coding; pair them with confident coders.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☑2 ☐3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☑C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 10 (25-29 Oct 2026) - Period 2	Ch 8 - 8.7 Iteration: count-controlled loops

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 10 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Task 8.2: 80% submitted; test tables better. Today: FOR...TO...NEXT loops, STEP, and Python range().

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students write count-controlled loops (FOR ... TO ... STEP ... NEXT) in pseudocode and Python for/range(), and trace them. Syllabus 8.1.4(c); AO2.

Success criteria — *what will successful learning look / sound like?*

I can write a FOR loop with and without STEP; I can match it to Python range() correctly (end value!); I can trace a loop in a trace table.

Key knowledge / skill / understanding:

Count-controlled = fixed number of repeats; range(1, 11) gives 1-10; STEP changes the increment.

Likely misconception / access or challenge need:

range(1,10) includes 10; the loop variable can't be used inside; STEP must be 1.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Brainstorming **Time:** 6 min

What students will do:

List everyday tasks repeated a known number of times (5 daily prayers, 10 push-ups, 30 students' marks). Decide start, end and step for each.

Evidence tool: Mini-whiteboards

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Whiteboards.

Support (low performers / SEN):

Picture prompts.

Stretch (fast learners / gifted):

Find a task that is repeated an unknown number of times.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 22 min

What students will do:

Code along: times-table printer, countdown with STEP -1, sum of 1..N; then write pseudocode FOR loops and convert to Python.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.7; syllabus pseudocode guide p.45; Python.

Support (low performers / SEN):

Templates with range() hints.

Stretch (fast learners / gifted):

Print a multiplication grid using a loop inside a loop (nested preview).

ACTIVITY 3 *student task + evidence of learning*

Strategy: Problem solving **Time:** 9 min

What students will do:

Trace table challenge: trace a FOR loop with STEP 2 and an accumulator; check with the Python debugger.

Evidence tool: Exit ticket

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Trace table sheet; Thonny debugger.

Support (low performers / SEN):

Partially completed trace table.

Stretch (fast learners / gifted):

Predict the output if STEP is -2.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: write 'I will not talk in class' 100 times - or use a loop. Model FOR Count <- 1 TO 10 ... NEXT Count and Python for count in range(1, 11). Use Thonny's step debugger to watch the counter change.

What is new or innovative in this lesson? *(ETEC looks for this)*

Students watch loop variables change live in the debugger, connecting trace tables to real execution.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Off-by-one with range(); changing the loop variable inside; forgetting NEXT; indentation errors. Likely: 'my loop runs 9 times'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: write a FOR loop that outputs 5, 10, ... 50; give range() for 1 to 20. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.7 count-controlled questions; program: output the sum of the first N odd numbers.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Debugger made loops visible; trace tables improved. Students' everyday examples were rich and personal.

What will I change next time?

Explain range end value with a number line first; most errors were off-by-one.

Students needing follow-up:

10BR1: 2 students absent - send the loop worksheet. Gifted students ready for nested loops.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☑3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 11 (1-5 Nov 2026) - Period 1	Ch 8 - 8.7 Iteration: pre-condition loops

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 11 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Exit ticket: 78% wrote a correct FOR loop with STEP; range() end values weak. Today: WHILE...DO...ENDWHILE when repeats are unknown.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students write pre-condition loops (WHILE ... DO ... ENDWHILE) in pseudocode and Python while, and choose between count-controlled and pre-condition loops. Syllabus 8.1.4(c); AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I can write a WHILE loop that ends correctly; I can explain why it may run zero times; I can choose FOR or WHILE for a scenario.

Key knowledge / skill / understanding:

Condition tested before each repeat; loop may not run at all; the condition must change inside the loop.

Likely misconception / access or challenge need:

WHILE always runs at least once; infinite loops are syntax errors.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Inquiry / investigation **Time:** 8 min

What students will do:

Predict-then-run: 3 WHILE snippets (normal, never runs, infinite). Predict output, run, explain, learn to stop an infinite loop.

Evidence tool: Mini-whiteboards

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Snippets in Thonny.

Support (low performers / SEN):

Annotated snippets.

Stretch (fast learners / gifted):

Rewrite the infinite loop so it ends.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Problem solving **Time:** 22 min

What students will do:

Program: a savings tracker that adds a monthly amount WHILE Total < Goal and counts months; then a guessing game using WHILE until the guess is right.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.7; Python.

Support (low performers / SEN):

Flowchart of the savings loop.

Stretch (fast learners / gifted):

Add RANDOM target and limit guesses.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Debate **Time:** 7 min

What students will do:

FOR or WHILE? Teams argue for 4 scenarios (reading 30 marks, password entry, game loop, countdown).

Evidence tool: Oral questioning

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Scenario slide.

Support (low performers / SEN):

Sentence starters.

Stretch (fast learners / gifted):

Name a scenario where both work and which is clearer.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: 'Keep stirring WHILE the tea is too hot'. Model WHILE Total < Goal DO ... ENDWHILE and Python while; show how the condition variable changes; show Ctrl+C / Stop for infinite loops.

What is new or innovative in this lesson? *(ETEC looks for this)*

Savings goals use students' own targets (Umrah trip, new phone) to make iteration personal and numeric.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Condition never changes (infinite loop); initialising the variable inside the loop; off-by-one on month count. Likely: 'my program froze'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit check: write a WHILE loop that halves a number until it is below 1; explain zero-run case. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.7 pre-condition questions; program: count how many years to double money at 5% interest.

5 REFLECT — After the lesson *Complete within a day. This is reflective-practice evidence and feeds your next Starting point.*

What worked — and how do I know?

Savings goals were meaningful and varied by background; all classes built the tracker. Predict-then-run caused good surprise.

What will I change next time?

Teach stopping infinite loops before practice; 4 students lost work.

Students needing follow-up:

10BR3: 3 students struggle to initialise counters; small-group support next lesson.

COVERAGE CHECK *Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.*

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H

Create	Discussion	Pair / group	Learner choice	Self-assessment	Peer assessment	SEN / low	Gifted
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 11 (1-5 Nov 2026) - Period 2	Ch 8 - 8.7 Iteration: post-condition loops

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 11 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Exit check: 80% wrote a terminating WHILE loop; counter initialisation weak. Today: REPEAT...UNTIL and input validation loops.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students write post-condition loops (REPEAT ... UNTIL) in pseudocode, emulate them in Python (while True / break), and compare all three loop types. Syllabus 8.1.4(c); AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I can write REPEAT...UNTIL for a validation loop; I can explain it runs at least once; I can code it in Python and compare it with WHILE.

Key knowledge / skill / understanding:

Condition tested after the body; always runs at least once; UNTIL stops when condition is TRUE (opposite of WHILE).

Likely misconception / access or challenge need:

UNTIL condition works like WHILE condition; Python has a REPEAT keyword.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 10 min

What students will do:

Worked example: password entry REPEAT INPUT Password UNTIL Password = "Secret" (syllabus example), then Python while True/break.

Evidence tool: Mini-whiteboards

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Syllabus pseudocode p.45; Python.

Support (low performers / SEN):

Side-by-side pseudocode/Python card.

Stretch (fast learners / gifted):

Add an attempts counter that locks after 3 tries.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Stations / carousel **Time:** 20 min

What students will do:

Three stations (6-7 min): A) range-check validation (mark 0-100); B) menu that repeats until 'Q'; C) convert a WHILE loop to REPEAT and explain the difference.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Station cards; Python.

Support (low performers / SEN):

Station cards with code skeletons.

Stretch (fast learners / gifted):

At C, prove with a test where WHILE and REPEAT give different results.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Cooperative learning **Time:** 7 min

What students will do:

Groups complete a comparison table: FOR vs WHILE vs REPEAT (when tested, minimum runs, best use, example).

Evidence tool: Peer-assessment checklist

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Comparison table template.

Support (low performers / SEN):

Table with examples to sort.

Stretch (fast learners / gifted):

Add Python equivalent for each.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: an ATM asks for PIN at least once. Model REPEAT ... UNTIL with the syllabus layout and the inverted condition; then Python while True: ... if condition: break. Explain station rotation.

What is new or innovative in this lesson? *(ETEC looks for this)*

Validation stations connect Chapter 7 validation algorithms to real code; students build a class comparison table used for revision.

Progress recorded / shared:

☑ Gradebook ☑ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Inverted condition errors (UNTIL vs WHILE logic); break inside wrong block; validation not re-prompting. Likely: 'my loop stops immediately'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: write REPEAT...UNTIL for an age 11-18 range check; state minimum runs for each loop type. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.7 post-condition questions; complete loop comparison table in notebook.

5 REFLECT — After the lesson *Complete within a day. This is reflective-practice evidence and feeds your next Starting point.*

What worked — and how do I know?

Stations kept pace high; comparison tables were strong and shared on Teams. 75% completed all three stations.

What will I change next time?

Teach the UNTIL condition inversion with a visual (stop sign) before stations.

Students needing follow-up:

Students who found station C hard (approx. 3 per class) get a WHILE-REPEAT conversion sheet.

COVERAGE CHECK *Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.*

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H
Create	**Discussion**	**Pair / group**	**Learner choice**	**Self-assessment**	**Peer assessment**	**SEN / low**	**Gifted**
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 12 (8-12 Nov 2026) - Period 1	Ch 8 - 8.8 Totalling and 8.9 Counting

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 12 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Exit ticket: 82% wrote correct REPEAT validation. Today: totalling and counting inside loops, linking to Chapter 7 standard algorithms.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use totalling (Total <- Total + Value) and counting (Count <- Count + 1) inside loops, and calculate averages. Syllabus 8.1.4(d); AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I initialise totals and counters to 0 before the loop; I update them correctly inside the loop; I output an accurate total, count and average.

Key knowledge / skill / understanding:

Totalling adds values; counting adds 1 when a condition is met; initialise before the loop, output after it.

Likely misconception / access or challenge need:

Initialise inside the loop; average = total / loop counter even when skipping values.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Problem solving **Time:** 8 min

What students will do:

Count and total by hand: a list of 10 temperatures - how many above 30 and what is the total and average? Write the algorithm steps.

Evidence tool: Mini-whiteboards

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Data slide.

Support (low performers / SEN):

Calculator allowed for warm-up only.

Stretch (fast learners / gifted):

Explain why average must be REAL.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Project-based task **Time:** 22 min

What students will do:

Class charity sale program: input item prices until 0, output total, number of items, items over 20 SAR and average price; pseudocode first, then Python.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.8-8.9; Python.

Support (low performers / SEN):

Skeleton with loop and comments.

Stretch (fast learners / gifted):

Track the most expensive item (link to finding maximum).

ACTIVITY 3 *student task + evidence of learning*

Strategy: Think-pair-share **Time:** 7 min

What students will do:

Trace table of a counting/totalling algorithm with 5 inputs; compare with partner and fix differences.

Evidence tool: Exit ticket

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Trace table sheet.

Support (low performers / SEN):

Trace table with columns labelled.

Stretch (fast learners / gifted):

Add a column for a running average.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: the school charity sale - how much did we raise and how many items sold? Model Total <- 0, Count <- 0 before a loop, updates inside, OUTPUT after. Show Python equivalent and integer vs real average.

What is new or innovative in this lesson? *(ETEC looks for this)*

Real school charity context; the same program is designed, coded and traced, mirroring Paper 2 scenario-question skills.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Resetting total inside the loop; counting the sentinel 0; dividing by zero when no items; integer division for average. Likely: 'my total is always the last value'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit check: write the two lines that total and count; identify where initialisation belongs. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.8-8.9 questions; program: input 10 marks, output total, count of passes (>= 50) and average.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Charity context resonated; students added cultural items to the sale list. 80% got total, count and average right.

What will I change next time?

Show the sentinel-value bug live; several students counted the 0.

Students needing follow-up:

10BR2: 3 students reset totals inside loops; quick re-teach with a trace table.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☑C ☐H	☑1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H
Create	**Discussion**	**Pair / group**	**Learner choice**	**Self-assessment**	**Peer assessment**	**SEN / low**	**Gifted**
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 12 (8-12 Nov 2026) - Period 2	Ch 8 - 8.10 String handling

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 12 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Charity programs: 80% correct; sentinel counting errors fixed by most. Today: string handling LENGTH, SUBSTRING, UCASE, LCASE.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use LENGTH, SUBSTRING, UCASE and LCASE in pseudocode and Python (len, slicing, upper, lower), noting position 1 vs 0. Syllabus 8.1.4(e); AO1, AO2.

Success criteria — *what will successful learning look / sound like?*

I can predict the result of LENGTH, SUBSTRING, UCASE, LCASE; I can convert SUBSTRING (s, start, length) into Python slicing correctly; my username generator works.

Key knowledge / skill / understanding:

Strings are sequences of characters; pseudocode SUBSTRING starts at 1 in the syllabus examples, Python indexes from 0.

Likely misconception / access or challenge need:

SUBSTRING(s,1,5) = s[1:5] in Python; LENGTH counts only letters, not spaces.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*	ACTIVITY 2 *student task + evidence of learning*	ACTIVITY 3 *student task + evidence of learning*
Strategy: Inquiry / investigation **Time:** 10 min	**Strategy:** Problem solving **Time:** 20 min	**Strategy:** Gamification **Time:** 7 min
What students will do: Investigate in the Python shell: len(), slicing, upper(), lower() on students' own names (with consent); record results and derive rules.	**What students will do:** Username generator: first 3 letters of surname + first letter of first name + birth year last 2 digits, all lowercase; write pseudocode with SUBSTRING/LCASE, then Python.	**What students will do:** Kahoot 'String surgeon': predict outputs of 8 string expressions in pseudocode and Python.
Evidence tool: Portfolio / notebook work Assessment: ☑ Diagnostic ☐ Formative ☐ End Grouping: ☐ Individual ☑ Pair ☐ Group	**Evidence tool:** Practical / performance task Assessment: ☐ Diagnostic ☑ Formative ☐ End Grouping: ☑ Individual ☐ Pair ☐ Group	**Evidence tool:** Digital quiz (Forms / Kahoot / Quizizz) Assessment: ☐ Diagnostic ☑ Formative ☐ End Grouping: ☑ Individual ☐ Pair ☐ Group
Resources: Investigation sheet; Python shell.	**Resources:** Coursebook (CUP 9781108915144) 8.10; syllabus string functions p.42.	**Resources:** Kahoot.
Support (low performers / SEN): Sheet with examples completed.	**Support (low performers / SEN):** Worked SUBSTRING-to-slice conversion card.	**Support (low performers / SEN):** Questions with images.
Stretch (fast learners / gifted): Find how to get the last 3 characters in 2 ways.	**Stretch (fast learners / gifted):** Handle names shorter than 3 letters; add validation.	**Stretch (fast learners / gifted):** Bonus: write the expression for a given output.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: school email usernames - how are they made? Model LENGTH("Happy Days") = 10, SUBSTRING("Happy Days", 1, 5) = "Happy", UCASE/LCASE (syllabus examples), then Python len, s[0:5], upper(), lower().

What is new or innovative in this lesson? *(ETEC looks for this)*

Names from different languages (Arabic transliteration, hyphenated names) become test data, showing why robust string handling matters.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Off-by-one in slicing; forgetting strings are immutable; names with Arabic script or spaces/hyphens. Likely: 'SUBSTRING(1,5) = [1:5]'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: convert SUBSTRING(Word, 2, 3) into Python; predict UCASE("riyadh"). Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.10 questions; program: count vowels in a sentence (uses loop + string handling).

5 REFLECT — After the lesson *Complete within a day. This is reflective-practice evidence and feeds your next Starting point.*

What worked — and how do I know?

Using students' own names (with consent) made strings personal; transliterated names created great test cases. 80% converted SUBSTRING correctly.

What will I change next time?

Teach 1-based vs 0-based indexing with a visual ruler before coding.

Students needing follow-up:

10BR1: 3 students confused about slicing end index; extra practice sheet.

COVERAGE CHECK *Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.*

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☑2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 13 (15-19 Nov 2026) - Period 1	Ch 8 - 8.11 Nested statements

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 13 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

String exit ticket: 80% correct; slicing end index weak. Today: nested selection and iteration (up to 3 levels) before the break.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students write and trace nested statements: selection inside iteration, iteration inside iteration, up to three levels. Syllabus 8.1.5; AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I can trace a nested loop; I can write a loop with an IF inside it; my nested program is correctly indented and ends with ENDIF/NEXT in the right places.

Key knowledge / skill / understanding:

Nesting = a statement inside another; inner loop completes fully for each outer repeat; exam max 3 levels.

Likely misconception / access or challenge need:

Inner loop runs once per program; ENDIF and NEXT can be in any order.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Experiment / lab **Time:** 10 min

What students will do:

Nested loop lab: change the outer and inner ranges of a pattern program (stars, multiplication grid) and record how output changes.

Evidence tool: Observation checklist

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Pattern programs in Python.

Support (low performers / SEN):

Lab sheet with prediction boxes.

Stretch (fast learners / gifted):

Predict total iterations as outer x inner.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 20 min

What students will do:

Worked example then build: exam-results analyser - loop through students, IF mark >= 50 count pass ELSE count fail, plus a nested loop for 3 subjects each.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.11; Python.

Support (low performers / SEN):

Structure diagram showing nesting levels.

Stretch (fast learners / gifted):

Add a third level: CASE for grade letter inside the loops.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Jigsaw **Time:** 7 min

What students will do:

Groups each trace one nested algorithm, then explain the trace to another group.

Evidence tool: Peer-assessment checklist

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Nested algorithm cards; trace tables.

Support (low performers / SEN):

Trace tables with columns set up.

Stretch (fast learners / gifted):

Find the line that makes an algorithm inefficient.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: a clock - minutes loop inside hours loop. Model a nested FOR with an IF inside (syllabus example of row totals), then Python with indentation; colour-code levels on the board.

What is new or innovative in this lesson? *(ETEC looks for this)*

Colour-coded nesting levels and a pattern lab make abstract structure visible.

Progress recorded / shared:

☑ Gradebook ☑ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Mis-indented blocks; inner counter not reset; ENDIF placed after NEXT; more than 3 levels making code unreadable. Likely: 'my grid prints in one line'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit check: how many times does OUTPUT run in a 4 x 3 nested loop? Correct an indentation error. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.11 questions. Before the autumn break: revise loops and selection with the class comparison table.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Pattern lab was popular; students compared Islamic geometric patterns with code patterns. 75% wrote a correct nested program.

What will I change next time?

Colour-coded indentation helped; print it as a handout next time.

Students needing follow-up:

Students below expectations on loops (approx. 3 per class) get a revision pack over the break.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☐3 ☑C ☐H	☑1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 13 (15-19 Nov 2026) - Period 2	Ch 8 - 8.12 Subroutines: procedures and functions

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 13 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Nested exit check: 78% counted iterations correctly. Last lesson before the autumn break: procedures and functions.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students define and call procedures (PROCEDURE ... ENDPROCEDURE, CALL) and functions (FUNCTION ... RETURNS ... ENDFUNCTION) in pseudocode and Python def/return. Syllabus 8.1.6(a)-(b); AO1, AO2.

Success criteria — *what will successful learning look / sound like?*

I can explain procedure vs function; I can write and call a procedure with CALL; I can write a function that RETURNS a value and use it in an expression.

Key knowledge / skill / understanding:

Subroutines reduce repetition and aid testing; a function returns a value; functions are not called with CALL.

Likely misconception / access or challenge need:

Functions print their answer; CALL is used for functions; defining a subroutine runs it.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Discussion / dialogu **Time:** 7 min

What students will do:

Look at a long program with repeated code; discuss what could become a subroutine and why (maintenance, testing, teamwork).

Evidence tool: Oral questioning

Assessment: ☑ Diagnostic ☐ Formative ☐ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Repetitive program printout.

Support (low performers / SEN):

Highlighted repeated blocks.

Stretch (fast learners / gifted):

Name a real app feature that would be a function.

ACTIVITY 2 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 21 min

What students will do:

Code along: PROCEDURE DrawLine(Size) with CALL; FUNCTION Area(Length, Width) RETURNS REAL; then refactor the results analyser using one procedure and one function.

Evidence tool: Practical / performance task

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.12; syllabus pseudocode p.46-47; Python.

Support (low performers / SEN):

Templates with def lines provided.

Stretch (fast learners / gifted):

Write a function IsValidMark(Mark) RETURNS BOOLEAN.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Think-pair-share **Time:** 8 min

What students will do:

Procedure or function? Pairs classify 6 subroutine descriptions and write the header line in pseudocode.

Evidence tool: Exit ticket

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Classification sheet.

Support (low performers / SEN):

Examples with keyword hints.

Stretch (fast learners / gifted):

Write the CALL or expression that uses each.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 7 min

Hook: a recipe that says 'make the sauce (see page 12)'. Model PROCEDURE Line(Size : INTEGER) and CALL Line(60); then FUNCTION with RETURNS and RETURN, and Python def/return. Stress: no CALL for functions.

What is new or innovative in this lesson? *(ETEC looks for this)*

Students refactor their own earlier program, seeing directly how subroutines make code shorter and easier to maintain.

Progress recorded / shared:

☑ Gradebook ☑ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

print instead of return; calling function without using its value; CALL used with functions; defining after use in Python. Likely: 'my function prints None'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: write a function header that returns the larger of two integers; explain one difference from a procedure. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Over the autumn break (optional): coursebook 8.12 questions 1-3; refactor one earlier program with a subroutine.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Recipe analogy worked across cultures (students shared family recipes). 75% wrote a working function; CALL vs function call clear for most.

What will I change next time?

Give extra practice on return vs print; it was the most common error.

Students needing follow-up:

Students who missed lessons this week: share code notes on Teams before the break.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☑1 ☐2 ☐3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☑3 ☐C ☐H
Create	**Discussion**	**Pair / group**	**Learner choice**	**Self-assessment**	**Peer assessment**	**SEN / low**	**Gifted**
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☐2 ☑3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 15 (29 Nov-3 Dec 2026) - Period 1	Ch 8 - 8.12 Subroutines: scope and parameters

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 15 / Lesson 1 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Back from autumn break: retrieval quiz on loops and subroutines first. Before the break 75% wrote working functions; return vs print shaky.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use parameters (up to three) and distinguish local and global variables, explaining scope. Syllabus 8.1.6(b)-(c); AO1, AO2.

Success criteria — *what will successful learning look / sound like?*

I can pass up to three parameters to a subroutine; I can predict which variables are local or global; I can explain why local variables are safer.

Key knowledge / skill / understanding:

Parameters pass values in; local variables exist only inside the subroutine; global variables are accessible everywhere.

Likely misconception / access or challenge need:

Changing a local variable changes the global one with the same name; more parameters is always better.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*	ACTIVITY 2 *student task + evidence of learning*	ACTIVITY 3 *student task + evidence of learning*
Strategy: Gamification **Time:** 8 min	**Strategy:** Inquiry / investigation **Time:** 20 min	**Strategy:** Debate **Time:** 9 min
What students will do: Retrieval quiz after the break (Quizizz): 10 questions on loops, selection and subroutines.	**What students will do:** Scope investigation: run 4 Python programs with same-named local and global variables; predict, run, explain. Then write a function Discount(Price, Percent, IsMember) with 3 parameters.	**What students will do:** 'Global variables should be banned.' Teams argue using maintainability and testing evidence; class vote.
Evidence tool: Digital quiz (Forms / Kahoot / Quizizz) Assessment: ☑ Diagnostic ☐ Formative ☐ End Grouping: ☑ Individual ☐ Pair ☐ Group	**Evidence tool:** Portfolio / notebook work Assessment: ☐ Diagnostic ☑ Formative ☐ End Grouping: ☐ Individual ☑ Pair ☐ Group	**Evidence tool:** Oral questioning Assessment: ☐ Diagnostic ☑ Formative ☐ End Grouping: ☐ Individual ☐ Pair ☑ Group
Resources: Quizizz.	**Resources:** Coursebook (CUP 9781108915144) 8.12; scope programs; Python.	**Resources:** Debate prompt.
Support (low performers / SEN): Read-aloud mode.	**Support (low performers / SEN):** Scope diagram (boxes inside boxes).	**Support (low performers / SEN):** Sentence starters.
Stretch (fast learners / gifted): Explain the reason for 2 answers.	**Stretch (fast learners / gifted):** Use the global keyword and explain why it is discouraged.	**Stretch (fast learners / gifted):** Give a case where a global constant is acceptable.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Re-entry after the break: routines, success criteria. Model parameters with a function call and a scope diagram (global box containing subroutine boxes). Show local variable 'disappearing' after return.

What is new or innovative in this lesson? *(ETEC looks for this)*

Students investigate scope experimentally and then debate a real software-engineering practice.

Progress recorded / shared:

☑ Gradebook ☐ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

Expecting local changes to affect globals; wrong order of arguments; too many parameters; missing return. Likely: 'my global didn't change'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Exit ticket: identify local vs global in a 6-line program; write a 3-parameter procedure header. Learners' questions.

Homework / platform *(e.g. Madrasati)*

Coursebook 8.12 scope questions; program: function that returns the average of three parameters.

5 REFLECT — After the lesson

Complete within a day. This is reflective-practice evidence and feeds your next Starting point.

What worked — and how do I know?

Retrieval quiz showed good retention after the break (avg 72%). Scope experiments clarified a hard concept; debate was lively.

What will I change next time?

Allow more time for re-settling after the break; shorten debate.

Students needing follow-up:

Students below 50% on retrieval quiz (approx. 3 per class): targeted revision session.

COVERAGE CHECK

Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.

Literacy read + write · 2-1-1-6	Numeracy numbers / data · 2-1-1-6	Digital students use it · 2-1-1-4	Digital research trusted sources · 2-1-1-4	Real-life ① application · 2-1-1-5	Real-life ② application · 2-1-1-5	Inquiry experiment / research · 2-1-1-7	Thinking problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H
Create something new · 2-1-1-7	**Discussion** expressing ideas · 2-1-1-8	**Pair / group** cooperative work · 2-1-1-8	**Learner choice** independent work · 3-2-1-6	**Self-assessment** 2-2-1-1	**Peer assessment** 2-2-1-1	**SEN / low** specific support · 2-1-1-3	**Gifted** specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H	☐1 ☐2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☑C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback

MRISB 4-STEP LESSON DESIGN | UbD × ETEC

OUTCOME → PROOF → PREPARE → RESPOND → REFLECT *Plan the learning first; let the checklist follow. Click a field to type · click a box to tick it.*

Teacher	Subject	Grade / Section	Date / Period	Unit / Topic
Mr. Ashraf Elbialy	Computer Science (IGCSE 0478)	Grade 10 - Classes 10BR1 to 10BR3	Wk 15 (29 Nov-3 Dec 2026) - Period 2	Ch 8 - 8.13 Library routines + 8.14 Maintainability

Pacing (2-1-1-2): ☑ On plan ☐ Ahead ☐ Behind

Weeks ahead / behind: 0 **Week / lesson:** Week 15 / Lesson 2 of 2

Starting point (2-2-1-3): *What did my last check (quiz, exit ticket, homework) show — and what am I changing today because of it?*

Scope exit ticket: 75% identified local vs global. Final lesson of Term 1: library routines and making programs maintainable, then a mini-review.

1 OUTCOMES — What must students learn?

Desired learning outcome(s) — *trace to the curriculum / standard and the table of specifications (ToS)*

Students use library routines MOD, DIV, ROUND, RANDOM and make programs maintainable with meaningful identifiers, comments and subroutines. Syllabus 8.1.7-8.1.8; AO2, AO3.

Success criteria — *what will successful learning look / sound like?*

I can use ROUND and RANDOM correctly in pseudocode and Python; I can improve an unreadable program with identifiers, comments and a subroutine; I can explain each improvement.

Key knowledge / skill / understanding:

Library routines are pre-written (ROUND, RANDOM); maintainable code is easy for others to read and update.

Likely misconception / access or challenge need:

RANDOM() returns whole numbers; comments make programs run slower; short identifiers are better.

2 PROOF — What will students do to show they have learned it?

Three activities, each with a DIFFERENT strategy, resource and evidence tool. Then tick the Coverage Check on page 2.

ACTIVITY 1 *student task + evidence of learning*

Strategy: Modelling / worked exam **Time:** 10 min

What students will do:

Dice simulator: Value <- ROUND(RANDOM() * 5, 0) + 1 (syllabus style), then Python random/round; discuss RANDOM range.

Evidence tool: Mini-whiteboards

Assessment: ☐ Diagnostic ☑ Formative ☐ End

Grouping: ☑ Individual ☐ Pair ☐ Group

Resources:

Syllabus p.42; Python random module.

Support (low performers / SEN):

Formula card for random ranges.

Stretch (fast learners / gifted):

Simulate 100 rolls and count each face (links to counting).

ACTIVITY 2 *student task + evidence of learning*

Strategy: Problem solving **Time:** 20 min

What students will do:

Maintainability makeover: a working but unreadable program (single-letter names, no comments, repeated code). Students refactor with meaningful identifiers, comments and one subroutine; mark against a checklist.

Evidence tool: Rubric

Assessment: ☐ Diagnostic ☐ Formative ☑ End

Grouping: ☐ Individual ☑ Pair ☐ Group

Resources:

Coursebook (CUP 9781108915144) 8.13-8.14; messy program file; rubric.

Support (low performers / SEN):

Checklist of 4 must-do improvements.

Stretch (fast learners / gifted):

Add a function and a constant; justify each change.

ACTIVITY 3 *student task + evidence of learning*

Strategy: Cooperative learning **Time:** 7 min

What students will do:

Groups build a revision mind map of all Chapter 8 topics taught this term and share it on Teams.

Evidence tool: Self-assessment checklist

Assessment: ☐ Diagnostic ☐ Formative ☑ End

Grouping: ☐ Individual ☐ Pair ☑ Group

Resources:

Mind map template; Teams.

Support (low performers / SEN):

Template with branch names.

Stretch (fast learners / gifted):

Link each branch to a past-paper question type.

3 PREPARE — What do I need to do so students can succeed?

Set students up to succeed:

☑ Hook / prior learning ☑ Explain or model ☑ Success criteria shared

☑ Key vocabulary / examples ☑ Real-life connection ☑ Clear instructions / resources

☑ Praise, recognition and reinforcement planned

My setup / modelling: Time: 6 min

Hook: two versions of the same program - which would you rather fix at 2 a.m.? Model library routines (DIV, MOD, ROUND, RANDOM) and the syllabus guidance on maintainable programs. Share rubric for the makeover.

What is new or innovative in this lesson? *(ETEC looks for this)*

A 'maintainability makeover' turns syllabus guidance into a practical refactoring task; groups produce a shared term revision map.

Progress recorded / shared:

☑ Gradebook ☑ Parents ☑ Written feedback in notebook / portfolio

4 RESPOND — What will I do with what I see and hear?

If students struggle or finish early I will:

☑ Re-teach ☑ Regroup ☑ Give extra support

☑ Stretch ☑ Have students redo / improve

What I will watch for / likely responses:

RANDOM range errors; ROUND with no places; refactoring that breaks the program (test after each change). Likely: 'comments are only for the teacher'.

Close / exit check *(summative + time for learners' questions)* Time: 7 min

Self-assessment (Forms): confidence for each Chapter 8 topic + one target for Term 2 (arrays and file handling next). Learners' questions.

Homework / platform *(e.g. Madrasati)*

Revision: past-paper Paper 2 questions on Topic 8.1 (selected); upload refactored program with comments.

5 REFLECT — After the lesson *Complete within a day. This is reflective-practice evidence and feeds your next Starting point.*

What worked — and how do I know?

Makeover task showed real understanding of maintainability; groups' mind maps were strong revision tools.
Self-assessments matched teacher judgements.

What will I change next time?

Give the makeover file earlier; some groups ran short of time for testing.

Students needing follow-up:

Students with low self-assessment confidence (approx. 3 per class): revision clinic before Semester 1 exams.

COVERAGE CHECK *Tick where each must-have happens (1 · 2 · 3 = activity, C = close, H = homework). An empty box is a gap to fix before teaching. Literacy and numeracy apply in every subject.*

Literacy	Numeracy	Digital	Digital research	Real-life ①	Real-life ②	Inquiry	Thinking
read + write · 2-1-1-6	numbers / data · 2-1-1-6	students use it · 2-1-1-4	trusted sources · 2-1-1-4	application · 2-1-1-5	application · 2-1-1-5	experiment / research · 2-1-1-7	problem-solving · 2-1-1-7
☐1 ☐2 ☑3 ☑C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☑1 ☑2 ☐3 ☐C ☑H	☐1 ☐2 ☐3 ☐C ☑H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☐2 ☐3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H

Create	Discussion	Pair / group	Learner choice	Self-assessment	Peer assessment	SEN / low	Gifted
something new · 2-1-1-7	expressing ideas · 2-1-1-8	cooperative work · 2-1-1-8	independent work · 3-2-1-6	2-2-1-1	2-2-1-1	specific support · 2-1-1-3	specific stretch · 2-1-1-3
☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☐C ☐H	☐1 ☑2 ☑3 ☐C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☐1 ☐2 ☑3 ☑C ☐H	☐1 ☑2 ☐3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H	☑1 ☑2 ☑3 ☐C ☐H

ALWAYS VISIBLE IN DELIVERY: Safe, respectful climate • every learner drawn in • purposeful use of time • smooth transitions • learner voice • specific feedback and next steps • students act on feedback